Master 2 SD

Nom : COELHO FERNANDES
Prénom : Rodrigo César

Machine Learning in Sequences

Séance de TP 4

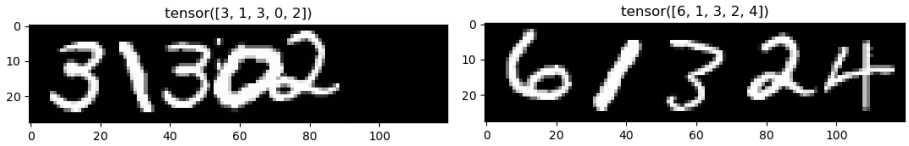
Embedded Training NN, RNN and CNN with the CTC Loss

1- Preparing data with pytorch

- **Question 1 : Que réalise la fonction pad_collate ?**

This function is the method used to perform padding on the sequences so that they have the same dimensions. The function receives the training batch and returns four elements.

The first two outputs are tensors containing all the examples with the same dimension, after performing a padding operation to adjust their sizes, and the same procedure is applied to their corresponding GTs. Below we can see an example of two sequences of different lengths adjusted to the same width using padding.



The outputs have different dimensions for the examples and the GT, being of dimensions B×T×i and T×B×i, respectively, where B is the batch size (the number of elements in the sequences), T is the length of the longest sequence, and **i** represents any number of dimensions.

The other two returned elements are tensors containing the lengths of all examples and their GTs. These lengths are used in the loss computation since it must be calculated according to the original lengths, not the adjusted ones after padding.

- **Question 2 : Que réalisent les lignes de code suivantes ? Expliquer l'organisation des données dans un lot.**

```
for batch in (train_dataloader):
    for i in range(5):
        plt.imshow(np.flip(batch[0][:,i,:].numpy().T,axis=0), cmap='gray')
        plt.title(batch[1][i])
        plt.show()

    break
```

The code above shows the first five examples of the batch, with the GT label as the title. We can state this because batch[0] represents a tensor of dimensions (sequence_length, batch_size, input_features). Meanwhile, batch[1] is a list of the labels for each example in the tensor, thus having a dimension equal to the batch_size.

2- Architecture du réseau de neurones

- **Question 3 : Quelle est l'architecture du réseau par défaut ? Quels sont les paramètres à modifier pour passer d'un réseau LSTM à un réseau BLSTM ?**

The network initially has 1 LSTM layer with 28 inputs (one for each row of the image) and generates 128 outputs, it is followed by a fully connected layer with 128 nodes. Para se utilizer um BLSTM é preciso alterar o valor da variavel 'blstm' para True. Also, When bidirectionality is enabled, the LSTM has 2 × hidden_size outputs features per timestep (concatenation of forward + backward), so a new correction to the number of features in the fully connected layer had to be made, as follows:

```
    # self.fc1 = nn.Linear(config['hidden_size'], config['hidden_size'])
    in_features = config['hidden_size'] * (2 if config['blstm'] else 1)
    self.fc1 = nn.Linear(in_features, config['hidden_size'])
```

- **Question 4 : Quels sont les paramètres à modifier pour créer un LSTM à deux couches ?**

In order to create a two-layer LSTM, only one variable has to be changed in the configurations of the model in the Main-CTC.py file:

```
        'lstm_layer':2,
```

- **Question 5 : Comment modifier ce code pour utiliser un MLP à une couche comme dans le TP précédent ?**

An MLP can be introduced between the LSTM and the CTC loss computation to improve feature processing while maintaining the LSTM's ability to model temporal dependencies.

3- Entrainement du réseau de neurones

- **Question 6 : dans la boucle d'apprentissage, expliquer les lignes de code suivantes. Pourquoi la log softmax doit-elle être utilisée ?**

```
        pred = torch.nn.functional.log_softmax(model(X.float()),dim = 2)
        loss = loss_fn(pred, y,X_l,y_l)
```

In first line, the result of the model call are raw scores and the log_softmax function is used to transform those scores in log probabilities, wich is important for numerical stability (because probabilities of sequences can become extremely small when multiplied together over many time steps) and compatibility with the following steps. While in the second line, the batch loss is calculated using those log probabilities and the GT.

- **Question 7 : Quelle est l'optimiseur utilisé par défaut ?**

The optimizer used is ADAM, as seen in the following line of the training section:

```
optimizer = torch.optim.Adam(my_lstm.parameters(),lr=config['learning_rate'])
```

- **Question 8 : Expliquer les étapes de la boucle principale d'apprentissage dans le code Main-CTC.py.**

Before the loop, the training and validation data are prepared into batches in the desired format of the previously discussed pad_collate function. Additionally, the model, loss function and optimizers are defined so that they can be used in the training process.

The main loop iterates for the desired number of epochs. It starts with the training and validation losses are calculated using the train_loop and valid_loop functions that iterate over all batches.

The training loop runs a forward prediction and computes the CTC loss to perform the backpropagation and try to improve the performance in the next batch. The average loss over all batches in this epoch is returned as a result, so it can be added to the list of losses over epochs. The validation loop function is similar, although it uses torch.no_grad() to avaid updating the gradients of the model in training.

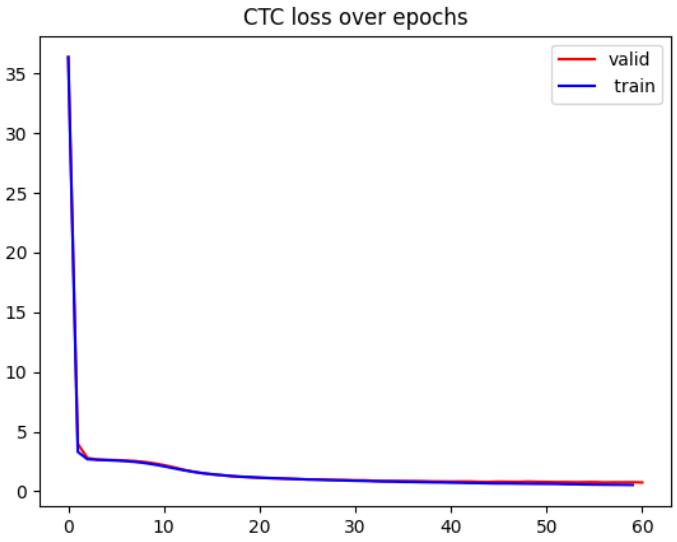
Finally, the main training loop plots the evolution of the loss over the epochs and do a checkpoint of the best model, to avoid a overfitted result in the end of training.

- **Question 9 : Compléter la fonction statesToSymbols()ci-dessous pour qu'elle réalise l'extraction de la séquence de symboles reconnus dans la meilleure séquence d'états prédite.**

```
def StatesToSymbols(best_path,T,config):
    last_symbol = best_path[0]
    best_symbols = [last_symbol]

    for t in range(1,T):
       if best_path[t] != last_symbol:
            last_symbol = best_path[t]
            best_symbols.append(last_symbol)

    bs = ''.join([str(best_symbols[i]) for i in range(len(best_symbols)) if
best_symbols[i] != config['blank_label']])

    return bs
```

4- Expérimentations

- **Question 10 : Observer l'allure typique de l'évolution de la loss CTC avec l'optimiseur ADAM, expliquer le phénomène.**



In the loss evolution graph, we can see the existence of two regimes: a first phase with a rapid decrease, followed by a second one where the decline is slower after a few epochs.

This happens because gradient descent reduces the average cross-entropy across all frames. Since the blank symbol is the most frequent one, the gradient descent primarily reduces the error of this majority class, leading to a quick improvement at the beginning.

Only when the blank errors become less dominant does the model begin to effectively minimize the recognition errors of the digit classes.

- **Question 11 : Compléter le tableau ci-dessous.**

Archi	N cellules	Batch_size	Test SER (%)	Test CER (%)
LSTM, 1 couche	128	32	92,0%	40,06%
	256	32	78,7%	26,66%
LSTM, 2 couches	128	64	100,0%	87,24%
	256	64	79,85%	28,50%
BLSTM, 1 couche	128	64	84,25%	31,45%
BLSTM, 2 couches	128	64	43,70%	11,70%
	256	64	39,65%	10,32%

From the obtained results, we can see that increasing the number of cells tends to improve model performance, as expected, though at the cost of higher processing time. The LSTM models achieved higher error rates than the BLSTM models for the same number of cells, which shows the latter's superior capacity to represent class complexity in this sequential context.

It is also observed that increasing the number of layers significantly reduced the error rate for the BLSTM, but not for the LSTM.

The extremely poor performance of the LSTM with 2 layers and only 128 cells is noteworthy. Possibly, the low number of cells was insufficient for each layer, compromising the final result.

- **Question 12 : Mettre en œuvre une architecture uniquement convolutive, puis réaliser des expériences mettant en évidence ses performances en les comparant aux performances de l'architecture LSTM et BLSTM.**

Archi	Batch_size	Test SER (%)	Test CER (%)
CNN	64	44,85%	12,09%

The fully convolutional architecture used was the one suggested in the correction file and can be seen below:

```
class CNN (nn.Module):
    def __init__(self, config):
        super(CNN, self).__init__()
        #premier bloc conv ReLU Pooling à 16 canaux = sortie de 16 X 14 X T/2
        self.DEVICE = config['device']
        self.cnn = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, stride=1,
padding=1),
            nn. ReLU(),
            nn.Conv2d(in_channels=16, out_channels=64, kernel_size=3,
stride=1, padding=1),
            nn. ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Conv2d(in_channels=64, out_channels=32, kernel_size=3,
stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(in_channels=32, out_channels=11, kernel_size=(7,1),
stride=1, padding=0),
            nn. Flatten (2) # on élimine la dimension verticale déjà ramenée à
1 par le kernel (7,1)
        )

    def forward(self, x):
        output = self.cnn(x)
        return output
```

From the obtained results, we can see that they are comparable to those achieved by the BLSTM with 2 layers and 128 cells (one of the most complex models used in the experiments).

In other words, even a simple convolutional architecture shows good performance on this dataset, since it is extremely efficient for image representation, which is precisely the nature of the analyzed dataset.

- **Question 13 : Reprendre la question précédente en mettant en œuvre une architecture hybride CNN-BLSTM. Analyser les performances et conclure.**

Archi	N cellules	Batch_size	Test SER (%)	Test CER (%)
CNN-BLSTM	256	64	15,10%	3,44%

In this final version, a convolutional architecture similar to the previous exercise was used, followed by a BLSTM with 2 layers and 256 cells, since this configuration achieved the best performance in the initial experiments. The architecture can be seen below:

```
class CNN_LSTM(nn.Module):
    def __init__(self, config):
        super(CNN_LSTM, self).__init__()
        self.config = config
        self.device = config['device']

        # === CNN ===
        self.cnn_features = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, stride=1,
padding=1),
            nn.ReLU(),
            nn.Conv2d(in_channels=16, out_channels=64, kernel_size=3,
stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Conv2d(in_channels=64, out_channels=32, kernel_size=3,
stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
```

```
        # === LSTM ===
        self.hidden_size = config['hidden_size']
        self.lstm_layer = config['lstm_layer']
        self.blstm = config['blstm']
        in_features = 32 * 7

        self.lstm = nn.LSTM(in_features,
                            self.hidden_size,
                            self.lstm_layer,
                            batch_first=False,
                            bidirectional=self.blstm)

        lstm_out_size = self.hidden_size * (2 if self.blstm else 1)
        self.fc = nn.Linear(lstm_out_size, config['num_classes'])

    def forward(self, x):

        features = self.cnn_features(x)
        B, C, H, T = features.size()

        features = features.permute(0, 3, 1, 2)
        features = features.reshape(B, T, C * H)
        features = features.permute(1, 0, 2)

        lstm_out, _ = self.lstm(features)
        out = self.fc(lstm_out)

        return out
```

The result shows a clear improvement in performance, with a CER of only 3.44%, demonstrating that both architectures can work well together due to their distinct specializations.

That is, the CNN effectively extracts image features, while the LSTM processes sequential data, together forming a well-adapted architecture for the context of this project.